人体图像多模态论文

March 18, 2026 at 08:02 PM

Gemini Conversation 8a38f5f2

2 conversation turns

Turn 1

User

> 现在确定stage1要怎么做吧，到底这五个任务怎么做。除了parsing都是文本做损失吗。parsing到底stage0怎么处理，以及适配stage1微调。参考上面几篇论文吧humanbench和sapiens，unihcp都是怎么做的。

Assistant

> 我们现在就把所有的线索收拢，为你敲定 **Stage 1 的终极落地蓝图**。
>
> 首先，极其明确地回答你的核心疑问：**是的，除了 Parsing（分割）之外，剩下的 4 个任务（Box, Pose, Attribute, Relation）在 Qwen-VL 这种大语言模型基座中，统统只使用“文本交叉熵损失 (Text Cross-Entropy Loss)”进行优化。** 它们在模型眼里没有任何区别，全都是在做“文字接龙”。
>
> 而 Parsing 是唯一的异类。结合你提到的 **HumanBench (UniHCP)** 和 **Sapiens** 的纯视觉思路，以及 VLM 的真实落地打法，我们来彻底把 Parsing 从 Stage 0 到 Stage 1 的处理方式理清楚。
>
> ---
>
> ## 一、 传统顶会是怎么处理 Parsing 的？（作为你的参考面）
>
> 在纯视觉架构中，Parsing 从来不用文本。
>
> - **HumanBench (UniHCP 架构):** 它的做法极其简单粗暴。图片经过 Backbone 和 Task Projector 后，对于 Parsing 任务，它接入了一个极简的解码头：两层 1×1 卷积加上 LayerNorm 和 ReLU。然后通过双线性插值放大到原图尺寸，直接算**像素级的交叉熵损失 (Pixel-wise Cross-Entropy Loss)**。
> - **Sapiens:** 类似地，用庞大的 ViT 提特征，然后用极简的线性层直接输出 28 个通道的概率图，算多分类交叉熵。
>
> **你的破局点：** 你现在用的是 Qwen-VL，你不能像 HumanBench 那样直接在视觉特征后接卷积头，因为你需要用 LLM 的“大脑”来理解用户的复杂指令。你需要把传统视觉的 Decoder，嫁接到 LLM 的隐层向量上。
>
> ---
>
> ## 二、 你的 Parsing 任务：Stage 0 (准备阶段) 到底要做什么？
>
> 在开始 Stage 1 微调之前，针对 Parsing 任务，你必须在 Stage 0 完成 **“架构缝合”** 和 **“数据格式化”**。
>
> **1. 词表扩充 (Tokenizer Expansion)** Qwen-VL 原本是不认识 `<SEG>` 这个词的。在 Stage 0，你必须调用 HuggingFace 的 `tokenizer.add_special_tokens({'additional_special_tokens':`

`['<SEG>']})`，把 `<SEG>` 作为一个独立的、不可分割的特殊 Token 加入词表，并同步扩大 Qwen-VL 的 Embedding 层矩阵。

2. 架构缝合 (Model Assembly) 在代码中定义你的完整模型类（`MyQwenVLWithMask`）：

- 加载并**冻结** DINOv3（作为你的高频像素特征提取器，提供极佳的边缘信息）。
- 定义一个轻量级的 MLP Decoder（参考 HumanBench 的设计，几层卷积或全连接即可）。

3. 数据准备 (Data Preparation) 你的 `train.jsonl` 里，Parsing 任务的数据必须长这样：

- **Prompt:** `"Segment the person's upper clothes."`
- **Answer:** `"Sure, here is the mask: <SEG>"`
- **Mask Path:** 对应一条本地 `.png` 单通道掩码图的路径。

三、 你的五个任务：Stage 1 (微调阶段) 到底怎么跑?

在 Stage 1，你的 DataLoader 会把这 5 个任务的数据混合在一起（像 HumanBench 证明的那样，混合多任务数据能学到极好的通用表征）。模型的前向传播和 Loss 计算如下：

任务 1-4：Box, Pose, Attribute, Relation

- **前向传播：** Qwen-VL 根据输入的图片和文本指令，自回归生成文本 Token（如坐标 `[100, 200...]` 或属性词 `male`）。
- **Loss 计算：** **只有 Text Cross-Entropy Loss**。模型将生成的 Token 与 `labels` 里的真实 Token 算交叉熵。外挂的 DINOv3 和 Decoder 在这些任务中**完全休眠，不参与计算**。

任务 5：Parsing (双轨制计算)

当 DataLoader 抽到一条 Parsing 数据时，真正的多模态协同开始了：

1. **文本轨计算 (Text Loss):** Qwen-VL 被迫输出 `"Sure, here is the mask: <SEG>"`。这里依然计算 **Text Cross-Entropy Loss**。这教会了模型“什么时候该触发画图”。
2. **向量拦截 (Vector Interception):** 当模型前向传播到 `<SEG>` 这个 Token 时，你的代码把最后一层对应 `<SEG>` 的 4096 维隐层向量（Hidden State）截获提取出来。
3. **像素轨计算 (Mask Loss):**
    - 将原图送入冻结的 DINOv3，提取出高分辨率特征图（Feature Map）。
    - 将 `<SEG>` 向量和特征图送入你的 MLP Decoder 进行点积/卷积解码。
    - Decoder 输出一张概率图。你拿这张概率图和真实的 `.png` 标签计算 **BCE Loss + Dice Loss**。
4. **反向传播 (Backward):**

- 文本 Loss 的梯度更新 Qwen-VL (通常通过 LoRA)。
- 像素 Loss 的梯度往回传，**只更新 MLP Decoder 和 `<SEG>` Token 的 Embedding 向量**，绝不干扰 Qwen-VL 的语言主体权重。

总结：你的 Stage 1 全景图

你的 Stage 1 本质上是一个 **“以文本为主，掩码为辅的联合训练”**：

- **四重文本任务** 帮你把 Qwen-VL 训成一个人体领域的“百事通”，掌握人体坐标、姿态常识和关系逻辑。
- **一重掩码任务** 帮你把 Qwen-VL 的 `<SEG>` Token 训成一个精确的“画笔遥控器”，并把你的外挂 Decoder 训成一个听话的“像素粉刷匠”。

这就完美适配了 Stage 1 的微调，并且架构上直接对标了目前 CVPR 最火的 LISA 范式。

我们现在的宏观架构和 Loss 走向已经完全敲定。**下一步，你是需要我为你写一段 PyTorch 代码，演示如何在 Forward 函数中精确“拦截并提取 <SEG> 向量”送给 Decoder 吗？** 这一步是整个代码工程里最容易卡脖子的地方。

Turn 2

Assistant

这张图把你的意图表达得非常清晰：**Stage 1 确实没有 DINOv3。它是对 Qwen-VL-3B 的视觉编码器（IE）和文本解码器（TD）进行端到端的联合微调（图上的两个蓝色“训”字就是铁证）。**

既然你要把这份方案交给 Codex 直接写代码，我们就抛弃所有虚无缥缈的概念。我为你整理了一份**“像素级精确、变量级定义”**的底层技术规格书（Technical Specification）。

以下是你需要在 Stage 1 实现的每一个细节。请直接将这份回答发给 Codex。

总体架构定义 (Architecture Definition)

在 Stage 1，Qwen-VL-3B 是唯一的模型。

- **Box、Pose、Attribute、Relation：** 这四个任务**100% 走纯文本生成**，通过 Text Cross-Entropy Loss 约束。
- **Parsing：** 借鉴 LISA 架构的精髓。由于没有 DINOv3，Parsing 必须从 Qwen-VL 自带的视觉编码器（ I_E ）中提取特征图，并与文本解码器（ T_D ）输出的特殊 `<SEG>` Token 隐层向量做**点积运算（Dot Product）**，从而解码出 Mask 图片。

Module 1: DataLoader 与数据格式化 (The Dataset)

Codex 需要写一个 `PyTorch Dataset`。一条训练数据同时包含文本和图片。

1. Tokenizer 扩充： 在初始化模型时，必须向 Tokenizer 添加特殊字符：

```
tokenizer.add_special_tokens({'additional_special_tokens': ['<SEG>']})
# 注意：扩充词表后，必须调用 model.resize_token_embeddings(len(tokenizer))
```

2. train.jsonl 的文本构建规范（Ground Truth Text）： 根据你图中的统一 Prompt，模型的完整输出文本（Target）必须被格式化为如下结构：

```
{
  "id": "001",
  "image": "path/to/img.jpg",
  "masks": ["path/to/shirt_mask.png", "path/to/pants_mask.png"],
  "conversations": [
    {
      "from": "user",
      "value": "请仔细描述图像中所有人物的位置、外观、属性、动作和关系。"
    },
    {
      "from": "assistant",
      "value": "位置：人物位于 <box>(250,300),(450,600)</box>。\n外观：上衣 <SEG>，裤子 <SEG>。\n属性：男性，青年，穿着红色上衣和蓝色裤子。\n动作：{\"nose\": [300, 260],
```

```
\"left_shoulder\": [280, 310]...}。\n关系：该男子正在骑自行车。"
    }
  ]
}
```

- **细节约束 1：** `labels` 张量的构建。用户 Prompt 部分必须被 Mask 掉（设为 `-100`），只有 Assistant 的回复参与 Text Loss 计算。
- **细节约束 2：** 如果有多个部位（如上衣、裤子），文本中就会出现多个 `<SEG>`。Dataset 必须按顺序读取对应的二值化掩码图（Binary Masks），并组合成一个 Shape 为 `[N, H_orig, W_orig]` 的张量返回，其中 N 是 `<SEG>` 的数量。

Module 2: Parsing Decoder 的网络结构 (The Model)

这是 Codex 编写模型 Forward 函数的核心。在没有 DINOv3 的情况下，解码器直接挂在 Qwen-VL 内部。

Codex 需要定义一个 `ParsingProjector`（一个极其轻量级的两层 MLP）：

```
 import torch.nn as nn
import torch.nn.functional as F

class ParsingProjector(nn.Module):
    def __init__(self, text_hidden_dim=3584, vis_dim=1024, out_dim=256):
        # text_hidden_dim 是 Qwen-VL 语言模型的隐藏层维度
        # vis_dim 是 Qwen-VL 视觉塔 (IE) 的特征维度
        super().__init__()
        # 1. 文本向量投影
        self.text_proj = nn.Linear(text_hidden_dim, out_dim)
        # 2. 视觉特征图投影 (类似于 HumanBench 的 1x1 卷积头)
        self.vis_proj = nn.Sequential(
            nn.Conv2d(vis_dim, out_dim, kernel_size=1),
            nn.GroupNorm(32, out_dim),
            nn.ReLU(),
            nn.Conv2d(out_dim, out_dim, kernel_size=1)
        )

    def forward(self, text_seg_embeddings, visual_feature_map):
        # text_seg_embeddings: [N, text_hidden_dim] (N是当前Batch中<SEG>的总数)
        # visual_feature_map: [B, vis_dim, H, W]

        # 转换维度
        v_seg = self.text_proj(text_seg_embeddings) # [N, out_dim]
        v_seg = v_seg.unsqueeze(-1).unsqueeze(-1)   # [N, out_dim, 1, 1]

        f_vis = self.vis_proj(visual_feature_map)   # [B, out_dim, H, W]

        # 注意：这里如果 N > 1 且 B = 1，需要做 broadcast 乘法 (点积)
```

```
        # mask_logits: [N, 1, H, W]
        mask_logits = torch.sum(v_seg * f_vis, dim=1, keepdim=True)
        return mask_logits
```

Module 3: 前向传播与特征拦截 (The Forward Pass)

Codex 在重写 Qwen-VL 的 `forward` 方法时，必须严格执行以下拦截逻辑：

1. 截获视觉特征图 (IE 的输出)：

- 图片输入 I_E 后，提取倒数第一层（或第二层）的特征序列。
- Qwen-VL 的 ViT 输出通常是序列 `[B, L, C]`。必须使用 `einops` 将其 Reshape 回 2D 特征图 `[B, C, H, W]`，备用。

2. 截获文本隐层向量 (TD 的输出)：

- 将文本和视觉特征送入 LLM。获取 LLM 最后一层的 `hidden_states`，Shape 为 `[B, Seq_Len, text_hidden_dim]`。
- 利用当前 Batch 的 `input_ids`，找到所有等于 `<SEG>` Token ID 的索引位置。
- 使用 `torch.gather` 或布尔索引，把这些位置的 `hidden_states` 提取出来，构成 `text_seg_embeddings` `[N, text_hidden_dim]`。

3. 生成 Mask Logits：

- 将上述两者送入 `ParsingProjector`，得到 `mask_logits` `[N, 1, H, W]`。
- 使用 `F.interpolate` 将其双线性上采样（Bilinear Upsample）到原图的真实尺寸（如 1024×1024）。

Module 4: 损失函数的严格定义 (The Losses)

Codex 需要实现两套并行的 Loss：

1. 文本损失 (约束 Box, Pose, Attr, Relation 和文本逻辑)：

```
 # shift_logits 和 shift_labels 逻辑同标准 LLM
loss_fct = CrossEntropyLoss(ignore_index=-100)
loss_text = loss_fct(shift_logits.view(-1, vocab_size), shift_labels.view(-1))
```

2. 掩码损失 (约束 Parsing Decoder)：

```
def calc_mask_loss(mask_logits, gt_masks):
    # mask_logits: [N, 1, H, W] (未经过 Sigmoid 的原始 logit)
    # gt_masks: [N, 1, H, W] (0 和 1 的浮点张量)
```

```
    # BCE Loss
    bce_loss = F.binary_cross_entropy_with_logits(mask_logits, gt_masks)

    # Dice Loss
    pred_masks = torch.sigmoid(mask_logits)
    intersection = torch.sum(pred_masks * gt_masks, dim=(2, 3))
    union = torch.sum(pred_masks, dim=(2, 3)) + torch.sum(gt_masks, dim=(2, 3))
    dice_loss = 1.0 - (2.0 * intersection + 1e-5) / (union + 1e-5)

    return bce_loss + dice_loss.mean()

loss_mask = calc_mask_loss(upsampled_mask_logits, ground_truth_masks)
```

3. 总损失：

```
loss = loss_text + loss_mask * mask_weight  # mask_weight 建议设为 1.0 或 2.0
```

给 Codex 的终极 Prompt 指令

你可以直接复制以下内容喂给 Codex：

"You are an expert PyTorch engineer. I am fine-tuning the Qwen-VL-3B model for a 5-task human-centric pipeline. Task: Box, Pose, Attribute, Relation are pure text generation tasks. Parsing is an instance-segmentation task driven by a <SEG> token routing mechanism.Action Required: Write a custom QwenVLForMultiTaskCausalLM class inheriting from the HuggingFace Qwen-VL base.Implement a ParsingProjector (a 1x1 Conv + ReLU based MLP) to act as the visual decoder.In the forward pass, intercept the visual feature map from the internal ViT encoder (IE). Reshape it to 2D (B, C, H, W).Identify the sequence positions of the <SEG> token in input_ids, extract their corresponding hidden states from the LLM decoder (TD), and pass both the hidden states and the visual feature map to the ParsingProjector to generate mask logits.Calculate loss_text (CrossEntropy ignoring -100) and loss_mask (BCE + Dice Loss). Return the combined loss. Ensure the code handles cases where N (number of SEG tokens) is variable per batch."

Exported from Gemini Voyager (https://github.com/Nagi-ovo/gemini-voyager) • 2 conversation turns
Generated on March 18, 2026 at 08:02 PM